Data Ingestion Sub-System – ETL Pipeline

Connecting flight delays, hotel availability, and live weather into one unified view

Omar Waseem · Revature Data Engineering · June 2026

Agenda

01

Problem Statement

The data gap stranded travelers face

02

Solution Overview

Unified ETL pipeline across three sources

03

Tech Stack

Python, PostgreSQL, Flask

04

Architecture

Source → Clean → Validate → Load → View

05

Implementation

Pipeline design and data quality

06

Live Demo

Dashboard walkthrough and test suite

07

Challenges & Solutions

Key obstacles and how we solved them

08

Future Enhancements

What's next for the pipeline

09

Q&A

Open discussion

Problem Statement

Your flight is delayed. You're stuck at an unfamiliar airport with no idea what to do next.

Wait or leave?

No way to know if the delay is 2 hours or overnight.

Hotel options?

Availability and pricing exist separately — not connected to flight data.

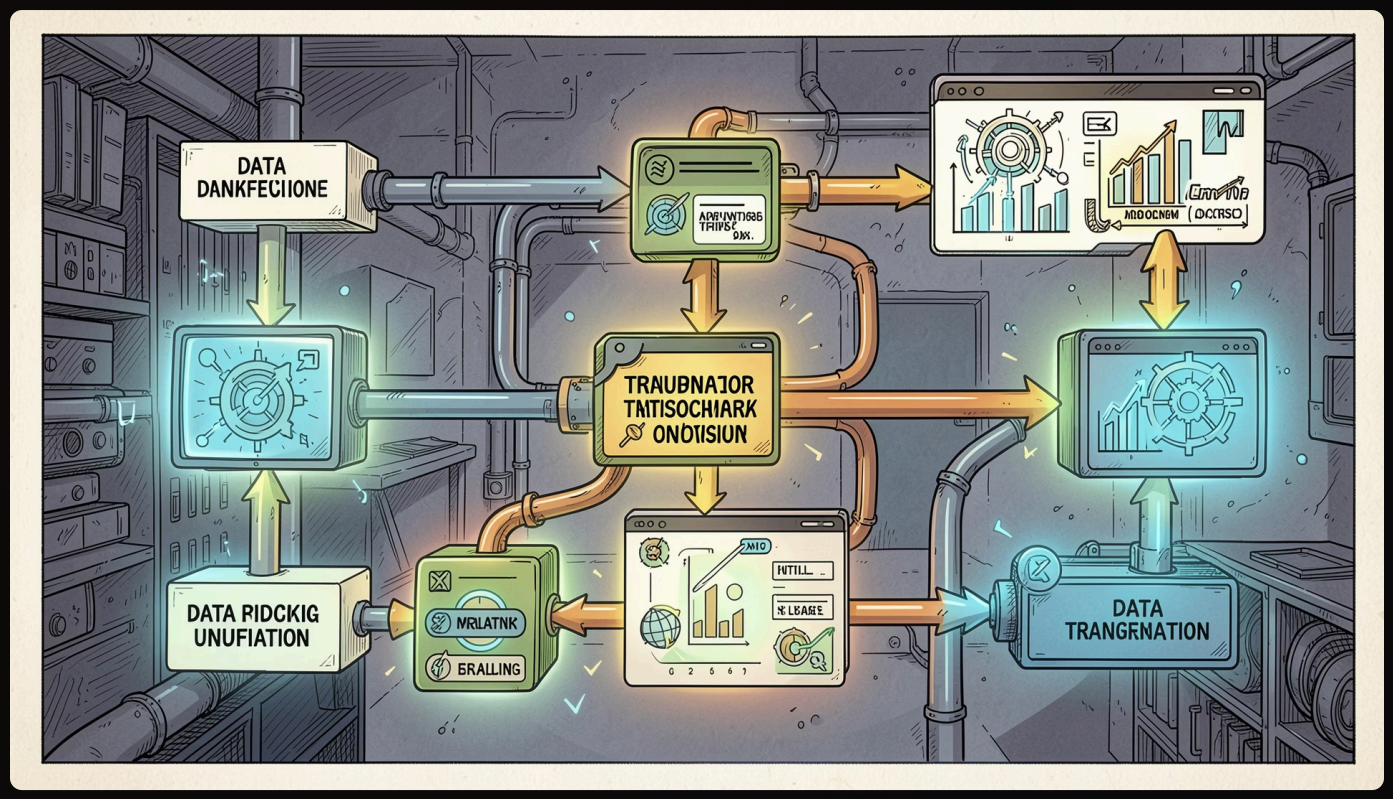
Weather conditions?

No live weather tied to your arrival city.

The data gap: Flight delay datasets, hotel bookings, and weather APIs exist in completely different formats — CSV, JSON, live APIs — with no standardized way to combine them. **This pipeline exists to close that gap.**

Solution Overview

A Python ETL pipeline that ingests all three data sources and unifies them into a single city-level view.



Three Sources, One Pipeline

Source	Format	Provides
Flight records	CSV	Delays, airlines, cities
Hotel bookings	JSON	Availability, pricing
Live weather	Open-Meteo API	Temp, wind, precipitation

All three sources are extracted, cleaned, validated, and loaded into PostgreSQL staging tables. A reporting VIEW (`vw_travel_summary`) joins all three into one city-level summary — and a Flask dashboard lets anyone look up any city and instantly see all three data points together.

Result: One pipeline run processes **4,500+ rows** across **98 cities** and produces a unified, query-ready dataset — the context a stranded traveler actually needs.

Tech Stack

Python 3

Core language — `pandas` for data processing, `psycopg2-binary` for PostgreSQL connectivity

PostgreSQL

Relational database for all staging tables and the `vw_travel_summary` reporting view

Open-Meteo API

Free, no-key-required live weather API — temperature, wind speed, and precipitation per city

Flask + Bootstrap 5

Dashboard with Chart.js visualizations — search any city and see all three data points instantly

pytest + pytest-cov

Unit and integration testing with coverage reporting — 33 tests at 87% coverage

PyYAML + python-dotenv

`sources.yml` for all source definitions and validation rules; `.env` for credentials

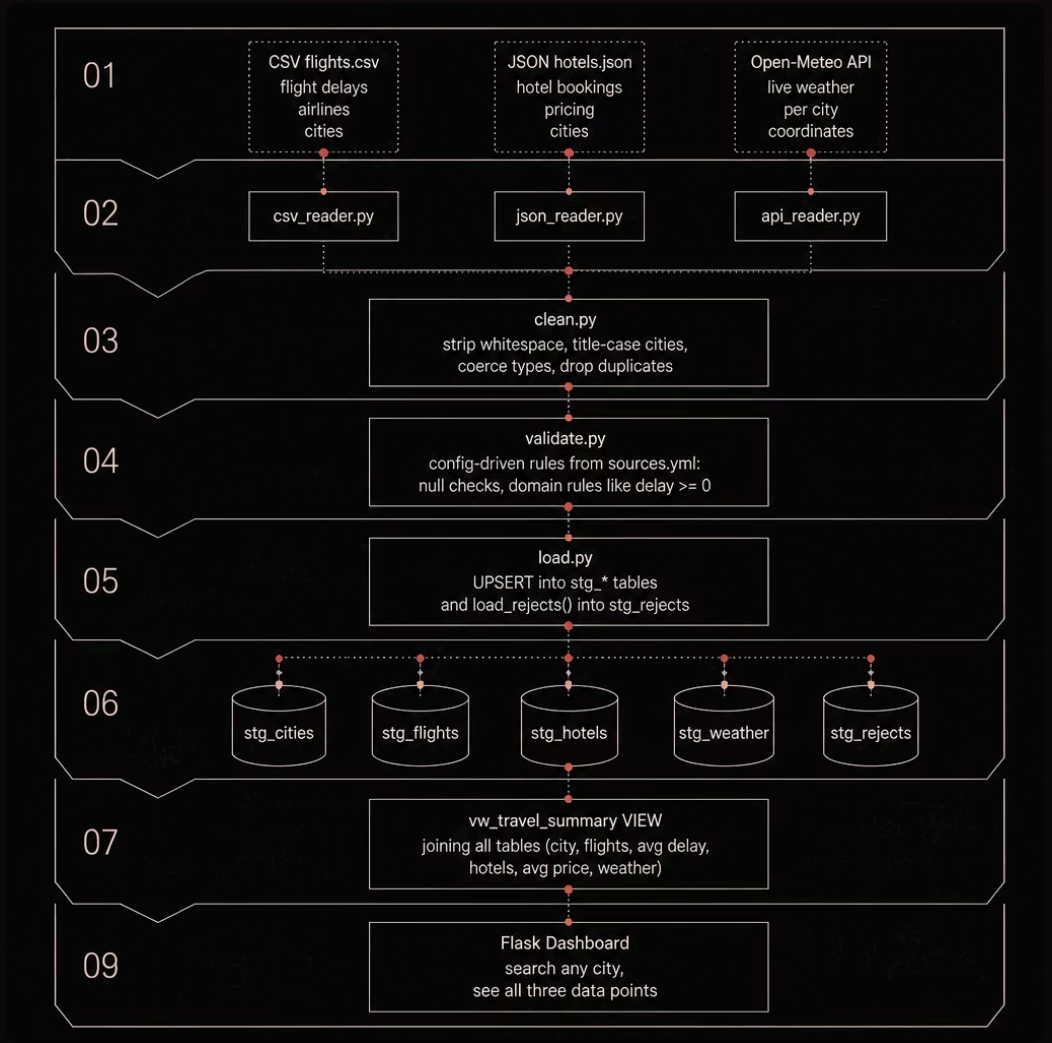
No ORMs — raw parameterized SQL throughout to prevent SQL injection and maintain full control over UPSERT logic.

Architecture

Data enters from three different sources at the top, each handled by its own reader. All three converge into the same **clean → validate → load** pipeline in the middle. Valid rows go into staging tables, invalid rows go into `stg_rejects`.

At the bottom, `vw_travel_summary` is a PostgreSQL VIEW that joins all three staging tables into one city-level row — which the Flask dashboard then queries and displays.

5 staging tables + 1 reporting view + 1 dashboard



```
.
./__pycache__
./__pycache__/conftest.cpython-314-pytest-9.0.3.pyc
./.coverage
./.env
./.gitignore
./.pytest_cache
./.pytest_cache/.gitignore
./.pytest_cache/CACHEDIR.TAG
./.pytest_cache/README.md
./.pytest_cache/v
./.pytest_cache/v/cache
./.pytest_cache/v/cache/lastfailed
./.pytest_cache/v/cache/nodeids
./config
./config/sources.yml
./conftest.py
./generate_data.sh
./presentation_outline.md
./README.md
./requirements.txt
./run_dashboard.sh
./run_pipeline.sh
./run_tests.sh
./session_notes.md
./src
./src/__pycache__
./src/__pycache__/clean.cpython-314.pyc
./src/__pycache__/config.cpython-314.pyc
./src/__pycache__/load.cpython-314.pyc
./src/__pycache__/log.cpython-314.pyc
./src/__pycache__/main.cpython-314.pyc
./src/__pycache__/validate.cpython-314.pyc
./src/clean.py
./src/config.py
./src/load.py
./src/log.py
./src/main.py
./src/readers
./src/readers/__pycache__
./src/readers/__pycache__/api_reader.cpython-314.pyc
./src/readers/__pycache__/csv_reader.cpython-314.pyc
./src/readers/__pycache__/json_reader.cpython-314.pyc
./src/readers/api_reader.py
./src/readers/csv_reader.py
./src/readers/json_reader.py
./src/validate.py
```

Implementation – Pipeline Design

Single source of truth: `config/sources.yml` defines each source, target table, and validation rule. New sources need **no code changes**.

readers/

One reader per source type: CSV, JSON, or API.

clean.py → validate.py → load.py

Shared flow for normalization, rules, and UPSERT loading.

airports.json

Central lookup for city coordinates and metadata.

Idempotent by Design

`ON CONFLICT DO UPDATE` keeps reruns safe and duplicate-free.

Implementation – Data Quality & Rejects

Validation Rules (from sources.yml)

Source	Column	Rule
flights	flight_id	not null
flights	delay_minutes	>= 0
hotels	guest_name	not null
hotels	price_per_night	> 0
airports	city, lat, lon	not null

Two reject types: **validation failures** (domain rule breaks) and **FK violations** (unknown city references). Every reject stored in `stg_rejects` with `source_name`, `raw_payload`, and `reason`.

Results

4,508 flights + 4,507 hotels + 98 weather records loaded cleanly.

Exactly **34 rejects** from intentional dirty rows — no silent data loss.

Live Demo

Run Pipeline

Execute ./run_pipeline.sh and stream structured logs.

Run Tests

Execute ./run_tests.sh — 33 tests passing, 87% coverage.

Explore Dashboard

Open http://127.0.0.1:5000 and view Travel Summary.

The core value of the pipeline is visible in the **Travel Summary** page — pick any city and immediately see its flights, average delay, hotel count, average price, and live weather all in one unified row.

Challenges, Enhancements & Q&A

City Name Casing Broke FK Integrity

`str.title()` transformed "Xi'an" → "Xi'An", causing 171 unexpected FK rejects. **Fix:** Applied the same transformation to airports so `stg_cities` stores matching casing.

Weather API Silently Skipping Cities

Case-sensitive lookup meant "Xi'An" didn't match "Xi'an" in the DB. **Fix:** Changed to `LOWER(city) = LOWER(%s)` and used the canonical name from the DB.

Three Independently Formatted Sources

CSV, JSON, and API with different schemas. **Fix:** `column_map` in `sources.yml` maps raw column names to a standardized internal schema.

🔭 Future Enhancements

- Schedule pipeline runs with Apache Airflow
- Add Kafka for real-time weather streaming
- Expand to rail and bus delay data
- Add unit tests for clean.py and validate.py

Thank you — open for questions.